Relational Databases and SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Exercise 1: Aggregations

Use the `university.db` created in the previous class (or recreate it using the provided solution).

1. Count how many students are in the database.
2. Count how many students are in the `Computer Science` department.
3. Calculate the average age of all students.
4. Find the minimum and maximum age in the database.
5. Show the number of students per department using `GROUP BY`.

Exercise 2: Subqueries and Advanced Joins

1. Find the names of students who are in a department that has more than 2 students.
2. Select departments that do not have any students (use a subquery with `NOT IN`).
3. Create a view named `student_details` that joins students and departments.

Exercise 3: Performance and Indexes

1. Explain the difference between a `Sequential Scan` and an `Index Scan`.
2. Create an index on the `name` column of the `students` table.
3. In SQLite, use `EXPLAIN QUERY PLAN` followed by a `SELECT` query to see if the index is being used.

```sql
EXPLAIN QUERY PLAN SELECT * FROM students WHERE name = 'Alice';
```

Exercise 4: Working with Large Datasets (Titanic)

1. Import the `dataset/titanic.csv` into a new SQLite database named `titanic.db`.
   - Hint: Use `.mode csv` and `.import dataset/titanic.csv passengers` inside sqlite3.
2. Write queries to answer:
   - What was the survival rate of the passengers?
   - What was the average age of survivors vs. non-survivors?
   - Which passenger class (`Pclass`) had the highest number of survivors?
   - Did women and children really have a higher chance of survival? (Group by Sex and a calculated Age category).

Exercise 5: Security Challenge (SQL Injection)

1. Look at this Python code:

   ```
   user_id = input("Enter user ID: ")
   cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")
   ```

2. What could a malicious user type to delete the entire `users` table?
3. Rewrite the code to be secure using a prepared statement.